

Restructured Question

The core technical question can be stated precisely as follows: if a program executes instructions sequentially, and a user has already configured a persistent preference (dark mode) before a page or app loads, why does the interface still briefly render in the wrong (light) state before correcting itself? The observation is that this "flash" happens consistently across different applications (Evernote) and different browsers (Edge, Chrome), suggesting a systemic cause rather than an isolated bug. The underlying assumption being tested is that clear instructions, given in advance, should allow a "translation layer" (the browser or app runtime) to render the correct state on the very first frame, with no intermediate incorrect state. This document explains why that assumption does not hold in current web and app architecture, and separates the concept of sequential code execution from the concept of visual rendering, which are related but not identical.

Program Execution Is Sequential; Rendering Is Not a Single Line of Code

The instruction that "a program executes one line of code at a time" describes how a single-threaded interpreter or CPU core processes instructions within a program, but it does not describe how a browser converts a full document into pixels on a screen. Rendering a web page is not one sequential operation; it is a multi-stage pipeline in which different resources (HTML, CSS, JavaScript, fonts, images) arrive and get processed at different times, often over a network connection with unpredictable latency^{[1][2]}. This pipeline is formally called the Critical Rendering Path (CRP), and it consists of the following major stages: parsing HTML into a Document Object Model (DOM), parsing CSS into a CSS Object Model (CSSOM), merging both into a render tree, computing layout, painting pixels, and compositing layers on the GPU^{[3][4]} ^[^2]. Because the DOM and CSSOM are built in parallel from separate downloaded resources, and JavaScript can interrupt DOM construction to fetch, compile, and run, the browser frequently reaches a paintable state before all styling information — including theme preference — has been applied^{[4][2]}.

This is the key correction to the mental model in the original question: the browser is not failing to follow clear instructions. It is following the instructions exactly as given, but the instructions themselves are typically split across multiple asynchronous steps rather than being available as a single unified command at the moment the first pixels are drawn^[^5].

Why "Already Logged In" and "Already Set to Dark Mode" Does Not Prevent the Flash

A stored preference (Evernote's saved theme, Chrome or Edge's OS-level dark mode setting, cached login session) exists as data, but that data has to be read and applied by code, and the timing of when that code runs relative to the first paint determines whether a flash occurs. In most web architectures, the sequence unfolds like this: the browser receives HTML with no theme class present, the CSS for default (often light) styling loads and applies, the browser paints the light-themed page, and only then does a JavaScript step read the stored preference from local storage or a cookie and apply the dark class, triggering a second paint^[^6]. The

visible "flash" is the gap between the first paint (light) and the second paint (dark) — typically 50 to 300 milliseconds, but perceptible especially against a mostly white background^{[5][6]}.

This explains the specific pattern described: "it renders dark and then I get this huge contrast effect where the whole screen turns white and then it turns black again." That sequence — dark, then white, then black again — indicates at least two separate rendering events layered on top of each other: an OS/browser-shell-level render (which may briefly default to white regardless of dark mode settings) followed by an application-level render (Evernote's own JavaScript-driven theme application)^[^7]. Being already logged in does not eliminate this, because authentication state and visual theme state are handled by completely separate systems; a valid session token tells the server who is asking for data, but it does not, by itself, control which CSS file or class gets applied first in the browser tab.

The Two Independent Layers Causing the Effect

Research into this exact phenomenon identifies two distinct and independent causes, which likely compound in the case being described.

Layer	What Causes the Flash	Why It Happens
OS / window manager (Windows Desktop Window Manager)	New browser windows briefly render with a default white background before the browser process takes over drawing	Windows does not know the intended theme of a newly created window until the application itself starts painting; it defaults to white as a placeholder ^[^7]
Application / web page (Evernote web, or any themed site)	Page loads with default light CSS, then JavaScript reads stored dark-mode preference and reapplies dark styling	Theme preference is often stored client-side (cookie/local storage) and can only be read and applied after some HTML/CSS has already rendered ^{[8][6]}

Microsoft has directly acknowledged the first cause: as of a Chromium browser engine patch, engineers modified the window-creation code so that a new window is "cloaked" (hidden), pre-filled with the dark background color, and only then made visible, specifically to eliminate the white flash reported by users on Chrome and Edge in dark mode^[^7]. This confirms that the issue the user observed is not imagined or isolated — it was formally identified, reproduced, and patched at the browser engine level, though the fix rolled out gradually and inconsistently across versions and platforms^{[7][9]}. Mozilla's own bug tracker contains a matching report describing the identical symptom in Firefox: "before the page loads, a white flash appears," confirmed as reproducible across multiple sites and unrelated to any single website's code^[^9].

Why This Was Not Caught Earlier and Why It Persists

The suspicion that "this is something that comes up before you even make it to the main page" and "how has nobody noticed this" is well-founded, but the more accurate framing is that it was noticed — repeatedly, by developers and end users across many products — and it remains difficult to fully eliminate for structural reasons rather than neglect. Several factors explain the persistence:

- **The preference is often stored client-side only:** Cookies or local storage are only readable after some part of the page has already started loading, so there is an inherent chicken-

and-egg problem unless the server itself renders the correct theme before sending HTML^[8]
^[6].

- **Render-blocking resources take priority over convenience fixes:** CSS is deliberately render-blocking by browser design (to avoid showing unstyled content), but that same blocking behavior does not automatically account for a user's saved theme unless engineers explicitly write code to check it before the first paint^{[1][3]}.
- **The fix requires deliberate architectural decisions, not passive testing:** The most reliable fix is to place an inline script directly in the HTML head that runs synchronously before the rest of the page renders, applying the correct theme class immediately^{[10][11][^8]}. If a design or engineering team does not implement this specific pattern, the flash reappears regardless of how much general usability testing occurs.
- **Third-party layers compound the problem:** caching systems, content delivery networks, and even OS-level window management (as shown by the Windows/Chromium case) sit outside a single app's control, meaning a flawless implementation by Evernote's engineers could still be undercut by how Chrome, Edge, or Windows itself handles the window before the app's code even runs^{[7][6]}.

This clarifies why usability testing does not automatically surface or fix the issue: testers evaluating "does dark mode work" typically check the resting state of a fully loaded page, not the 50-300 millisecond transition window during initial load or navigation, so the defect is easy to overlook unless testing specifically targets load-time and navigation-time rendering rather than steady-state appearance^[^5].

What "Correct" Execution Would Look Like

From a systems-design standpoint, eliminating the flash requires treating theme state as a rendering-critical input that must be resolved before the first paint, not as a cosmetic adjustment applied afterward. The most robust solutions identified in current engineering practice include:

- Applying the theme class to the root HTML element via a synchronous inline script placed in the document head, so it executes before the browser parses the rest of the page and before any stylesheet paints^{[10][11][^8]}.
- Rendering the correct theme server-side when the server has access to the user's stored preference, avoiding any client-side "guess first, correct later" pattern^{[8][6]}.
- Coordinating with OS-level window management so the window itself is filled with the correct background color before it becomes visible, which is the fix Microsoft implemented in Chromium for Windows^[^7].
- Increasing CSS specificity or load priority for dark-mode rules so that even if a brief unstyled state occurs, it resolves to the correct theme as quickly as possible rather than defaulting to light^[^6].

None of these fixes are exotic or unknown; they are documented, standard remediation patterns discussed across browser engineering blogs, bug trackers, and framework documentation. The persistence of the flash on some products and platforms reflects incomplete adoption of these patterns, competing engineering priorities, or trade-offs against

page-load speed, rather than a fundamental impossibility or an oversight born of not understanding the problem^{[10][7][11][8][5][6]}.

References

1. [Critical Rendering Path](#) - What blocks first paint and LCP on initial load — HTML, CSS, fonts, and scripts — and how to shorten...
2. [Understanding the Critical Rendering Path: How Browsers Build Web Pages](#) - Learn how browsers convert HTML, CSS, and JavaScript into rendered pixels. Understand the critical r...
3. [Critical Rendering Path Optimization | Browser Rendering](#) - The Critical Rendering Path (CRP) is the sequence of steps the browser must complete before it can p...
4. [Lesson 02 — The Critical Rendering Path \(Lead Lens\)](#) - How bytes become pixels — DOM → CSSOM → Render Tree → Layout → Paint → Composite — and the two resou...
5. [Dark Mode Flicker on Page Reload: How to Fix the White ...](#) - Dark mode has become a staple feature in modern web design, offering users reduced eye strain, impro...
6. [How to Fix White Screen Flashes \(FOUC\) When Loading WordPress ...](#) - The only way to eliminate the flash is to make the dark class present before Step 2, not after Step ...
7. [Microsoft finally fixing Chrome & Edge's white flash in dark ...](#) - Microsoft's switch to Chromium has benefited Chrome more than anything else. Google Chrome now has b...
8. [Fixing Dark Mode Flickering \(FOUC\) in React and Next.js - Rajiv](#) - Flickering in React and Next.js dark mode—also known as a flash of unstyled content (FOUC)—happens w...
9. [1863326 - In dark mode \(via Dark Reader extension\), a white flash appears when loading a new page - RESOLVED \(nobody\) in WebExtensions - Developer Outreach](#). Last updated 2023-12-11.
10. [What the FOUC? Dark mode with Astro transitions and Tailwind](#) - A Flash of Unstyled Content is when the browser renders the page before the CSS has been loaded. The...
11. [Dark Mode Pt. II: Flash of Unstyled Content \(FOUC\)](#).